In [ ]:
```
# EXPERIMENT - 8
## Data Pre-processing & Feature Engineering
PIN No.: 241UDS0111

Aim:** To preprocess a dataset by removing unnecessary features, scaling numerical features, and encoding
categorical variables using Python libraries.
```

In [ ]:
```
### Step 1 – Import the basic libraries
```

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
from sklearn.impute import SimpleImputer
```

In [ ]:
```
### Step 2 – Load & Analyze the Dataset
```

In [2]:
```
df_241UDS0111 = pd.read_csv("Experiment_8_241UDS0111_Retail_Customer_Features.csv")

# Dataset information
df_241UDS0111.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 101 entries, 0 to 100
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Customer_ID           100 non-null    str
 1   Age                   99 non-null     float64
 2   Annual_Income_k       100 non-null    float64
 3   Monthly_Visits        100 non-null    float64
 4   Average_Basket_Value  99 non-null     float64
 5   Gender                99 non-null     str
 6   Membership            99 non-null     str
 7   City                  100 non-null    str
 8   Preferred_Channel     100 non-null    str
 9   Is_Active             100 non-null    float64
 10  Satisfaction_Score    100 non-null    float64
dtypes: float64(6), str(5)
memory usage: 8.8 KB
```

In [3]:
```
# Display the first five records
df_241UDS0111.head()
```

Out[3]: Columns 1–8 of 12

	Customer_ID	Age	Annual_Income_k	Monthly_Visits	Average_Basket_Value	Gender	Membership
0	C1001	40.0	66.0	14.0	2066.84	Male	Silver
1	C1002	25.0	129.0	13.0	3227.10	Female	Silver
2	C1003	52.0	74.0	1.0	8324.87	Female	Basic
3	C1004	26.0	69.0	2.0	8091.84	Male	Platinum
4	C1005	52.0	89.0	8.0	8232.11	Female	Silver

Columns 9–12 of 12 · first column repeated

	City	Preferred_Channel	Is_Active	Satisfaction_Score
0	Rajahmundry	Store	0.0	4.0
1	Guntur	Website	1.0	3.0
2	Visakhapatnam	Website	1.0	2.0
3	Rajahmundry	Website	1.0	3.0
4	Visakhapatnam	Website	0.0	3.0

In [4]:
```
# Check the shape
df_241UDS0111.shape
```

Out[4]:
```
(101, 11)
```

In [5]:
```
# Check for null values
df_241UDS0111.isnull().sum()
```

Out[5]:
```
Customer_ID             1
Age                     2
Annual_Income_k         1
Monthly_Visits          1
Average_Basket_Value    2
Gender                  2
Membership              2
City                    1
Preferred_Channel       1
Is_Active               1
Satisfaction_Score      1
dtype: int64
```

In [6]:
```
# Remove completely empty rows
df_241UDS0111 = df_241UDS0111.dropna(how="all")
df_241UDS0111.shape
```

Out[6]:
```
(100, 11)
```

In [ ]:
```
### Step 3 — Remove Unnecessary Features
```

In [7]:
```
# Customer_ID is an identifier, not a useful numerical feature for this preprocessing exercise
df_241UDS0111 = df_241UDS0111.drop("Customer_ID", axis=1)

df_241UDS0111.columns
```

Out[7]:
```
Index(['Age', 'Annual_Income_k', 'Monthly_Visits', 'Average_Basket_Value',
       'Gender', 'Membership', 'City', 'Preferred_Channel', 'Is_Active',
       'Satisfaction_Score'],
      dtype='str')
```

In [ ]:
```
### Step 4 — Scale the Numerical Features
```

In [8]:
```
# Fill missing numerical values with the median before scaling
numeric_columns_241UDS0111 = [
    "Age", "Annual_Income_k", "Monthly_Visits",
    "Average_Basket_Value", "Satisfaction_Score"
]
numeric_imputer_241UDS0111 = SimpleImputer(strategy="median")
df_241UDS0111[numeric_columns_241UDS0111] = numeric_imputer_241UDS0111.fit_transform(
    df_241UDS0111[numeric_columns_241UDS0111]
)

# Standardize Age and Annual Income
z_scaler_241UDS0111 = StandardScaler()
df_241UDS0111[["Age", "Annual_Income_k"]] = z_scaler_241UDS0111.fit_transform(
    df_241UDS0111[["Age", "Annual_Income_k"]]
)

# View the dataset after StandardScaler
df_241UDS0111.head(3)
```

Out[8]:

Columns 1–8 of 11

	Age	Annual_Income_k	Monthly_Visits	Average_Basket_Value	Gender	Membership	City
0	-0.010062	-0.313347	14.0	2066.84	Male	Silver	Rajahmundry
1	-1.171099	1.403249	13.0	3227.10	Female	Silver	Guntur
2	0.918767	-0.095366	1.0	8324.87	Female	Basic	Visakhapatnam

Columns 9–11 of 11 · first column repeated

	Preferred_Channel	Is_Active	Satisfaction_Score
0	Store	0.0	4.0
1	Website	1.0	3.0
2	Website	1.0	2.0

In [9]:
```
mm_scaler_241UDS0111 = MinMaxScaler()
df_241UDS0111[["Monthly_Visits", "Satisfaction_Score"]] = mm_scaler_241UDS0111.fit_transform(
    df_241UDS0111[["Monthly_Visits", "Satisfaction_Score"]]
)

# View the dataset after MinMaxScaler
df_241UDS0111.head(3)
```

Out[9]: Columns 1–8 of 11

	Age	Annual_Income_k	Monthly_Visits	Average_Basket_Value	Gender	Membership	City
0	-0.010062	-0.313347	0.928571	2066.84	Male	Silver	Rajahmundry
1	-1.171099	1.403249	0.857143	3227.10	Female	Silver	Guntur
2	0.918767	-0.095366	0.000000	8324.87	Female	Basic	Visakhapatnam

Columns 9–11 of 11 · first column repeated

	Preferred_Channel	Is_Active	Satisfaction_Score
0	Store	0.0	0.75
1	Website	1.0	0.50
2	Website	1.0	0.25

In [ ]:
```
### Step 5 — Encode the Categorical Features
```

In [10]:
```
# Fill missing categorical values with the most frequent value
categorical_columns_241UDS0111 = ["Gender", "Membership", "City", "Preferred_Channel"]
categorical_imputer_241UDS0111 = SimpleImputer(strategy="most_frequent")
df_241UDS0111[categorical_columns_241UDS0111] = categorical_imputer_241UDS0111.fit_transform(
    df_241UDS0111[categorical_columns_241UDS0111]
)

# Use a separate LabelEncoder for each categorical column
label_encoders_241UDS0111 = {}
for col in ["Gender", "Membership"]:
    encoder_241UDS0111 = LabelEncoder()
    df_241UDS0111[col] = encoder_241UDS0111.fit_transform(df_241UDS0111[col].astype(str))
    label_encoders_241UDS0111[col] = encoder_241UDS0111

df_241UDS0111.head(3)
```

Out[10]: Columns 1–8 of 11

	Age	Annual_Income_k	Monthly_Visits	Average_Basket_Value	Gender	Membership	City
0	-0.010062	-0.313347	0.928571	2066.84	1	3	Rajahmundry
1	-1.171099	1.403249	0.857143	3227.10	0	3	Guntur
2	0.918767	-0.095366	0.000000	8324.87	0	0	Visakhapatnam

Columns 9–11 of 11 · first column repeated

	Preferred_Channel	Is_Active	Satisfaction_Score
0	Store	0.0	0.75
1	Website	1.0	0.50
2	Website	1.0	0.25

In [11]:
```
df_241UDS0111 = pd.get_dummies(
    df_241UDS0111,
    columns=["City", "Preferred_Channel"],
    drop_first=True,
    dtype=int
)

df_241UDS0111.head(3)
```

Out[11]: Columns 1–8 of 15

	Age	Annual_Income_k	Monthly_Visits	Average_Basket_Value	Gender	Membership	Is_Active
0	-0.010062	-0.313347	0.928571	2066.84	1	3	0.0
1	-1.171099	1.403249	0.857143	3227.10	0	3	1.0
2	0.918767	-0.095366	0.000000	8324.87	0	0	1.0

Columns 9–15 of 15 · first column repeated

	Satisfaction_Score	City_Kakinada	City_Rajahmundry	City_Vijayawada	City_Visakhapatnam	Preferred_Channel_Store	Preferred_Channel_Website
0	0.75	0	1	0	0	1	0
1	0.50	0	0	0	0	0	1
2	0.25	0	0	0	1	0	1

In [12]:
```
# Convert the Is_Active boolean-like values to integers (0/1)
df_241UDS0111["Is_Active"] = df_241UDS0111["Is_Active"].astype(int)

df_241UDS0111.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 14 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Age                        100 non-null    float64
 1   Annual_Income_k            100 non-null    float64
 2   Monthly_Visits             100 non-null    float64
 3   Average_Basket_Value       100 non-null    float64
 4   Gender                     100 non-null    int64
 5   Membership                 100 non-null    int64
 6   Is_Active                  100 non-null    int64
 7   Satisfaction_Score         100 non-null    float64
 8   City_Kakinada              100 non-null    int64
 9   City_Rajahmundry           100 non-null    int64
 10  City_Vijayawada            100 non-null    int64
 11  City_Visakhapatnam         100 non-null    int64
 12  Preferred_Channel_Store    100 non-null    int64
 13  Preferred_Channel_Website  100 non-null    int64
dtypes: float64(5), int64(9)
memory usage: 11.1 KB
```

In [13]:
```
# Check the final shape
df_241UDS0111.shape
```

Out[13]: (100, 14)

In [14]:
```
# Save the preprocessed dataset
df_241UDS0111.to_csv(
    "Experiment_8_241UDS0111_Processed_Retail_Customer_Features.csv",
    index=False
)
print("Preprocessed dataset saved successfully.")
```

```
Preprocessed dataset saved successfully.
```

In [ ]:
```
### RESULT
The retail customer dataset was preprocessed by removing an unnecessary identifier, handling missing values,
scaling numerical features using StandardScaler and MinMaxScaler, encoding categorical features using LabelEncoder
and one-hot encoding, and converting the boolean feature into integers. The processed dataset is saved as a CSV
file.
```